Aula 03 — Seleção de Modelos e Validação Cruzada

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §1.4–1.5; ISLP cap. 5

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- explicar por que o erro de treino é um estimador *otimista* do risco;
- descrever *data splitting* (treino/validação) e estimar o risco com a parte de validação;
- definir *validação cruzada* LOOCV e k-dobras e relacioná-las;
- discutir o balanço viés–variância *na própria estimativa do risco* e a escolha de k;
- usar a validação cruzada para **selecionar hiperparâmetros** sem vazamento;
- descrever o *bootstrap* para quantificar incerteza de uma estimativa.

Esta é a aula *operacional* mais importante do curso. Nas Aulas 01 e 02 apareceram hiperparâmetros — o grau do polinômio, o λ do Lasso — e nos dois casos a resposta para "como escolher?" foi "na Aula 03". Chegamos nela. A mesma ferramenta vai escolher o k do KNN, a profundidade da árvore, o C da SVM. Comece pela pergunta:

se eu ajustar um polinômio de grau 50 a 50 pontos, qual o erro no treino?
e isso quer dizer que é um bom modelo?

O erro é *exatamente zero* — 51 parâmetros passam por 50 pontos sem esforço — e o modelo é péssimo. Um critério de qualidade que dá nota máxima ao pior modelo possível não serve como critério. Precisamos de outro.

1 O problema: estimar o risco

Da Aula 01, o risco de uma função de predição g é $R(g) = \mathbb{E}[L(Y, g(\boldsymbol{X}))]$, esperança sobre uma observação *nova*. Para *selecionar* um modelo (escolher entre candidatos $g \in \mathcal{G}$, ou escolher um hiperparâmetro) precisamos comparar riscos — mas $R(g)$ é desconhecido porque depende de P. Logo, **tudo se resume a estimar bem o risco**.

1.1 Por que não usar o erro de treino?

O candidato ingênuo é o *erro quadrático médio no treino* (risco empírico):

$$\mathrm{EQM}(g) = \frac{1}{n}\sum_{i=1}^{n} \big(Y_i - g(\boldsymbol{X}_i)\big)^2. \tag{1}$$

O problema: g foi *escolhida para ajustar exatamente esses pontos*. O EQM(1) é um estimador **otimista** (enviesado para baixo) de $R(g)$, e esse viés — o *otimismo* — cresce com a complexidade do modelo.

Atenção: dois "vieses" diferentes, e eles andam em sentidos opostos

A palavra *viés* acabou de mudar de sentido, e vale parar um instante, porque a Aula 01 ensinou o outro:

- **o viés do modelo**, da decomposição viés–variância da Aula 01: $\mathbb{E}_{\mathcal{D}}[\hat{r}(\boldsymbol{x}_0)] - r(\boldsymbol{x}_0)$, o quanto o método erra o alvo r. Esse **cai** com a complexidade — é a curva que desaba a partir do grau 5 na figura da decomposição;
- **o viés do EQM de treino como estimador do risco**: $\mathbb{E}[\text{EQM}(g)] - R(g)$, o quanto a *régua* mente. Esse **cresce** com a complexidade, e é dele que a frase acima fala.

São objetos de níveis diferentes: um é o erro do modelo, o outro é o erro da *medida* do erro do modelo — e não há contradição em o mesmo modelo ter os dois. O grau 49 da §3 da `Aula prática 03` é o caso extremo: erro de treino 0,1029, *menor* que o 0,3580 do grau 5, e risco $2{,}5 \times 10^{13}$. Ao subir o grau, o viés do modelo desapareceu e o da régua explodiu junto. É por isso que o erro de treino não pode arbitrar entre modelos de complexidades diferentes: quanto mais flexível o candidato, mais a régua o favorece.

Atenção

Levado ao extremo, minimizar o erro de treino leva ao superajuste perfeito: um modelo flexível o bastante *interpola* os dados (erro de treino zero, em aritmética exata — ver a §3 da `Aula prática 03` para o que o ponto flutuante faz com isso) e generaliza pessimamente. **Nunca** selecione modelos nem reporte desempenho com o erro de treino.

Observação 1.1. Penalização (AIC, BIC) é uma alternativa: estima-se $R(g) \approx \text{EQM}(g) + P(g)$, com $P(g)$ crescente na complexidade (p.ex. $P(g) = 2p\,\hat{\sigma}^2/n$ no AIC). Funciona, mas exige um modelo probabilístico e contagem de "parâmetros" — nem sempre disponível (KNN, florestas). A validação cruzada é *agnóstica ao modelo* e por isso a preferimos.

2 Data splitting (conjunto de teste)

A ideia mais simples: separe os dados *aleatoriamente* em treinamento (p.ex. 70%) e teste (p.ex. 30%):

$$\underbrace{(\boldsymbol{X}_1, Y_1), \ldots, (\boldsymbol{X}_n, Y_n)}_{\text{treinamento}} \,, \; \underbrace{(\boldsymbol{X}_{n+1}, Y_{n+1}), \ldots, (\boldsymbol{X}_{n+m}, Y_{n+m})}_{\text{teste}} .$$

Daqui para a frente n é o tamanho do *treinamento* e m o do teste. Ajuste g *só* no treinamento e estime o risco *só* no teste:

$$\widehat{R}(g) = \frac{1}{m} \sum_{i=n+1}^{n+m} \big(Y_i - g(\boldsymbol{X}_i)\big)^2 . \tag{2}$$

Como o teste não foi usado para estimar g, (2) é um estimador *consistente* de $R(g)$ pela lei dos grandes números. A proporção também não é arbitrária: treinar um modelo exige mais informação do que avaliá-lo, e por isso a fatia maior fica com o treinamento.

Atenção: o *aleatoriamente* não é decoração

Se o banco vier ordenado por alguma variável — por data, por classe, por quem coletou —, cortar pelos índices originais produz dois conjuntos que não são amostras da mesma distribuição, e a estimativa fica sem sentido. Quem montou o banco pode ter ordenado por um motivo que você desconhece. Em `scikit-learn` isso é o `train_test_split(..., shuffle=True)`, que já é o padrão — mas vale saber por que existe.

Atenção: Limitações do data splitting

1. **Desperdício de dados:** g é treinada com apenas n das $n+m$ observações que você coletou; com poucos dados isso prejudica o ajuste.
2. **Variabilidade:** a estimativa depende da divisão sorteada — outra semente dá outro número. Pode ser instável.

A validação cruzada resolve os dois usando *todo o conjunto de treinamento* — e é importante ver o que ela *não* substitui. A divisão treino/teste continua sendo o primeiro passo, e o teste continua guardado até o fim. O que a validação cruzada dispensa é um *segundo* corte, o que separaria uma parte do treino para escolher o modelo: esse papel passa a ser feito pelas dobras, girando dentro do treino.

3 Validação cruzada

3.1 Leave-one-out (LOOCV)

Deixe *uma* observação de fora por vez. Seja g_{-i} o modelo ajustado sem a i-ésima observação. O estimador é

$$\widehat{R}_{\mathrm{LOO}}(g) = \frac{1}{n}\sum_{i=1}^{n}\big(Y_i - g_{-i}(\boldsymbol{X}_i)\big)^2. \tag{3}$$

Cada Y_i é predito por um modelo que *não* o viu. LOOCV é *aproximadamente não-viesado* para o risco esperado e não tem aleatoriedade de divisão.

Observação 3.1 (por que "aproximadamente não-viesado")**.** A conta é curta, e o "aproximadamente" tem uma causa precisa — vale ver as duas. Primeiro uma notação: escreva R_m para o risco *esperado* do procedimento treinado com m observações, isto é, a média de $R(g)$ também sobre o sorteio do conjunto de treino. Repare na diferença: $R(g_{-i})$ é o risco *daquele* ajuste, que depende dos dados que caíram nele; R_m é a média desses ao longo de todos os treinos possíveis de tamanho m. É de R_n que queremos uma estimativa.

Você tem n observações, e g_{-i} é ajustado com as outras $n-1$; chame esse conjunto de $\mathcal{D}_{-i}$. Todo o argumento está numa observação só: $(\boldsymbol{X}_i, Y_i)$ **não participou** do ajuste de g_{-i}. Dado $\mathcal{D}_{-i}$, o par é uma observação nova, independente do que treinou o modelo — e a definição de risco dá, literalmente,

$$\mathbb{E}\big[(Y_i - g_{-i}(\boldsymbol{X}_i))^2 \,\big|\, \mathcal{D}_{-i}\big] = R(g_{-i}).$$

Tome agora a esperança sobre $\mathcal{D}_{-i}$, que é um treino de tamanho $n-1$: $\mathbb{E}[R(g_{-i})] = R_{n-1}$, e o mesmo valor para todo i, porque os n conjuntos $\mathcal{D}_{-i}$ têm todos o mesmo tamanho e a mesma distribuição. Somando as n parcelas de (3),

$$\mathbb{E}\big[\widehat{R}_{\mathrm{LOO}}\big] = \frac{1}{n}\sum_{i=1}^{n}\mathbb{E}\big[(Y_i - g_{-i}(\boldsymbol{X}_i))^2\big] = \frac{1}{n}\sum_{i=1}^{n} R_{n-1} = R_{n-1},$$

exatamente — sem aproximação nenhuma. E note o que *não* foi preciso supor: em momento nenhum os g_{-i} precisaram ser independentes entre si. Não são, aliás — dois deles compartilham $n-2$ observações. Esperança é linear e não se importa; a dependência entre as parcelas vai afetar a *variância* do estimador, e é exatamente esse o assunto da §3.4.

O "aproximadamente", então, não está na conta: está no **alvo**. O LOOCV é não viesado para R_{n-1}, e você queria R_n — o risco do modelo que vai de fato usar, treinado com a amostra inteira. Como treinar com mais dados costuma ajudar, $R_{n-1} \geq R_n$, e o LOOCV tende a ser levemente *pessimista*. Levemente mesmo: o descompasso é de uma observação em n, pequeno demais para a simulação da Figura 3 separar de zero (lá o viés medido na LOOCV é $-0{,}01$, do tamanho do próprio ruído de Monte Carlo). É por isso que a LOOCV é a menos enviesada das validações cruzadas.

Atenção: se você for ler a Observação 1.5 do [AME]

O livro faz a mesma conta, mas partindo de um conjunto com $n+1$ observações, e conclui $\mathbb{E}[\,\cdot\,] = R(g)$ na bala. Não é outra conta nem um resultado mais forte: é a mesma, reindexada. Ele escolhe o tamanho do banco de modo que cada g_{-i} treine com exatamente n pontos, e aí o alvo sai limpo. O preço é que o banco da conta tem $n+1$ observações e o seu tem n — foi para não pagar esse preço que fizemos acima com n, deixando o descompasso aparecer onde ele de fato mora, no alvo.

O custo aparente é alto (n ajustes), mas nem sempre:

Proposição 3.2 (Atalho do LOOCV para modelos lineares)**.** *No MQO, nos* splines *e no Ridge com λ fixo, vale*

$$\widehat{R}_{LOO} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{Y_i - \widehat{g}(\boldsymbol{X}_i)}{1-h_{ii}}\right)^2,$$

onde $\widehat{g}$ é ajustado com todos *os dados e h_{ii} é o i-ésimo elemento diagonal (*leverage*) da matriz $\boldsymbol{H}$ que leva as respostas $Y_1,\dots,Y_n$ aos valores ajustados $\widehat{g}(\boldsymbol{X}_1),\dots,\widehat{g}(\boldsymbol{X}_n)$. No MQO, $\boldsymbol{H} = \boldsymbol{X}(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top$ é a matriz de projeção, e nos* splines *vale o mesmo, com as funções da base nas colunas de $\boldsymbol{X}$; no Ridge, a inversa do meio é a da solução fechada da Aula 02, e $\boldsymbol{H}$ deixa de ser uma projeção.*

Ou seja, um *único* ajuste basta. Esta é uma das fórmulas mais elegantes da área: o LOOCV "de graça". Vale notar o que ela diz: o resíduo de validação é o resíduo de treino *inflado* pelo fator $1/(1-h_{ii})$, e h_{ii} mede o quanto a observação i influencia sua própria predição. Pontos de alta alavancagem são justamente aqueles cujo resíduo de treino mais engana.

3.2 Validação cruzada com k dobras

Divida *o conjunto de treinamento* aleatoriamente em k lotes (dobras) disjuntos de tamanho $\approx n/k$, com índices $L_1,\dots,L_k$. O conjunto de teste saiu na divisão anterior e não entra em dobra nenhuma. Para cada j, treine g_{-j} *sem* o lote L_j e avalie nesse lote:

$$\widehat{R}_{k\text{-CV}}(g) = \frac{1}{n}\sum_{j=1}^{k}\sum_{i\in L_j}\big(Y_i - g_{-j}(\boldsymbol{X}_i)\big)^2. \tag{4}$$

Cada observação é usada para validação *exatamente uma vez* e para treino $k-1$ vezes. Quando $k=n$ recuperamos o LOOCV (3). Valores típicos: $k=5$ ou $k=10$.

Atenção: a mesma fórmula, escrita de dois jeitos

Se você abrir o [ISLP] — e a leitura recomendada pede que abra —, vai encontrar a validação cruzada definida de outro jeito. A fórmula (5.3) de lá calcula um EQM_j *dentro* de cada dobra e tira a média dos k:

$$\text{CV}_{(k)} = \frac{1}{k}\sum_{j=1}^{k}\text{EQM}_j, \qquad \text{EQM}_j = \frac{1}{|L_j|}\sum_{i\in L_j}\big(Y_i - g_{-j}(\boldsymbol{X}_i)\big)^2.$$

Não são fórmulas rivais. Agrupe (4) dobra a dobra e ela vira

$$\widehat{R}_{k\text{-CV}} = \sum_{j=1}^{k}\frac{|L_j|}{n}\,\text{EQM}_j\,:$$

a média dos *mesmos* EQM_j, só que **ponderada pelo tamanho das dobras**. Quando k divide n todas têm $|L_j| = n/k$, os pesos são todos iguais a $1/k$, e as duas contas dão o

mesmo número — exatamente o mesmo, não "aproximadamente". A diferença de fundo é o que cada uma toma como unidade: (4) pesa igualmente cada *observação*, a do [ISLP] pesa igualmente cada *dobra*.
Quando k *não* divide n elas divergem, e vale saber que o `scikit-learn` segue a do [ISLP] — `cross_val_score(...).mean()` é a média não ponderada. Como as dobras do `KFold` nunca diferem entre si em mais de uma observação, a divergência encolhe como k/n: a §6 da `Aula prática 03` mede 0,9% com $n = 50$ e $k = 7$, e nos 21 mil pontos do `superconductivity.csv` seria de 0,03%. Pequena — mas não inofensiva em amostra pequena: lá mesmo, escolher o grau do polinômio por uma ou por outra dá respostas diferentes em 9 de 300 amostras. A moral não é que uma das duas esteja certa; é que, com n pequeno, **convém escolher um k que divida n**, e aí a pergunta não se coloca.

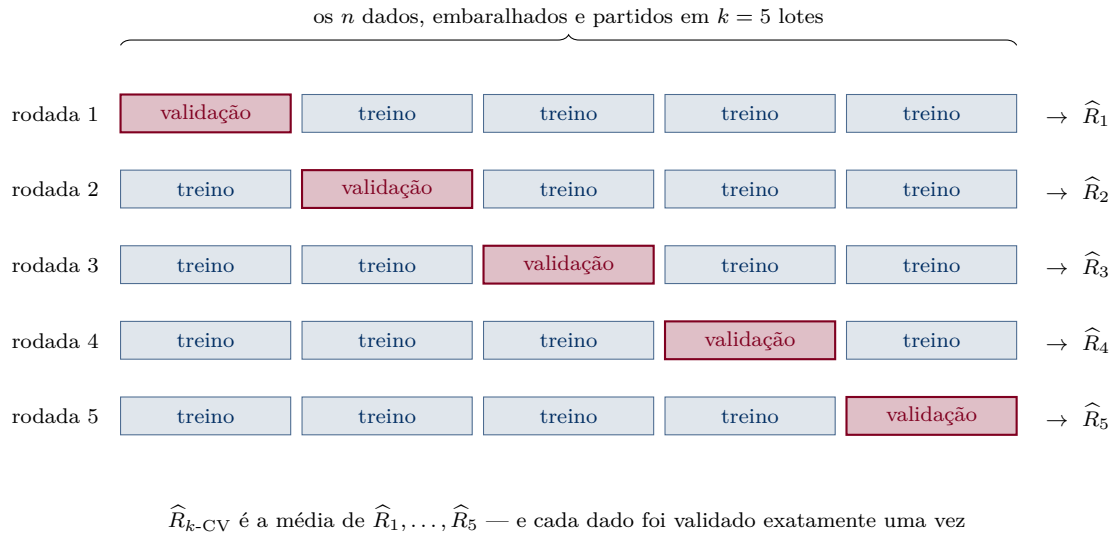


Figura 1: Validação cruzada com $k = 5$ dobras. Em cada rodada um lote diferente fica de fora do ajuste e serve para avaliar. Ao final, *toda* observação foi usada para validar (uma vez) e para treinar ($k - 1$ vezes) — é assim que a validação cruzada resolve o desperdício de dados do *data splitting*.

Ideia central

A validação cruzada estima o risco *de um procedimento* ("ajustar este tipo de modelo a dados como estes"), não de um g específico. Por isso, depois de escolher o hiperparâmetro por CV, **reajustamos o modelo final usando todos os dados**.

3.3 A validação cruzada funciona? Uma verificação

Dá para conferir, porque na população sintética da Aula 01 nós conhecemos o risco verdadeiro. A Figura 2 põe lado a lado o que o analista realmente tem (uma amostra de $n = 50$ pontos e a CV calculada nela) e o que ele nunca terá (o risco verdadeiro, obtido simulando 300 amostras).

Ideia central: o que a CV precisa acertar

Repare que a curva azul não coincide com a vinho: a CV *subestima* um pouco o risco. Isso não é problema. Para **selecionar** um modelo, o que importa é *onde* está o mínimo, não o valor no mínimo. O [ISLP] faz essa observação explicitamente: em três exemplos diferentes as curvas de CV ora subestimam, ora superestimam o risco, mas todas identificam corretamente o nível de flexibilidade certo. Se você quer *reportar* o desempenho, aí sim

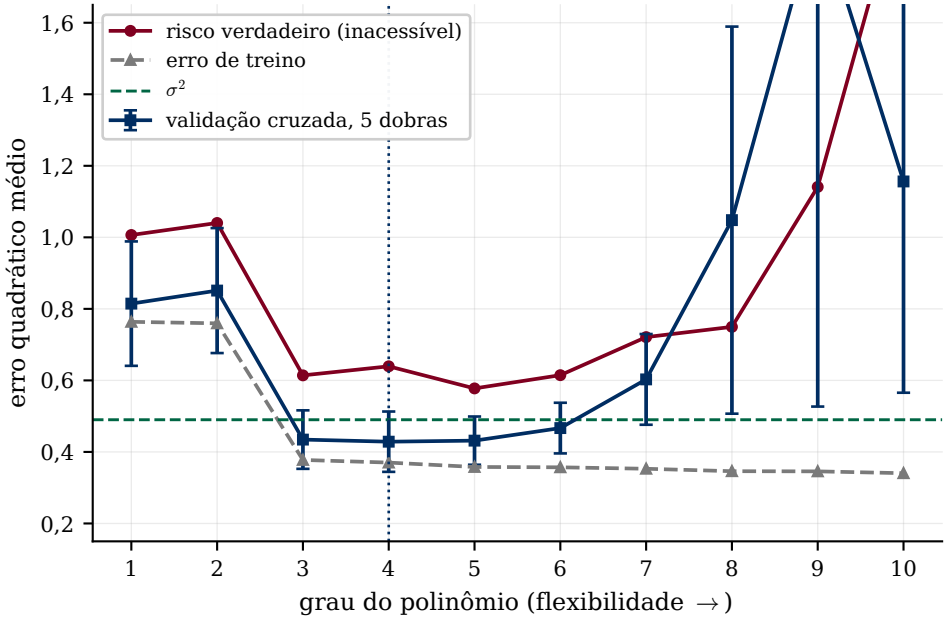


Figura 2: Validação cruzada de 5 dobras calculada sobre *uma única* amostra de $n = 50$, contra o risco verdadeiro e o erro de treino. As barras são de um erro-padrão. A CV reproduz a forma em "U" e localiza o mínimo no grau 4, contra o grau 5 do risco verdadeiro — e, o mais importante, não se deixa enganar pelo erro de treino, que continua descendo. Note também as barras se abrindo à direita: a CV *avisa* quando está insegura.

precisa do valor — e para isso usa-se um conjunto de teste separado, ou CV aninhada.

3.4 Viés, variância e a escolha de k

O argumento clássico para a escolha de k é ele mesmo um balanço viés–variância *da estimativa do risco*:

- k **grande (LOOCV):** cada g_{-j} treina com $\approx n$ dados $\Rightarrow$ *pouco viés*. Em contrapartida os n modelos são quase idênticos entre si e suas predições, muito correlacionadas — e a média de quantidades correlacionadas não se beneficia da lei dos grandes números como a de quantidades independentes. O argumento conclui: *variância alta*.
- k **pequeno (p.ex. 5):** cada g_{-j} treina com bem menos dados $\Rightarrow$ *mais viés*, no sentido pessimista: o procedimento avaliado não é bem o que você vai usar no final. Em troca, as dobras são mais independentes.

Ideia central: de onde vem o viés, e por que ele é sempre pessimista

A Observação 3.1 já explica o viés do segundo item, e a explicação vale para todo k. Com k dobras de tamanho igual, cada g_{-j} treina com $n - n/k$ observações, e o par $(\boldsymbol{X}_i, Y_i)$ de uma dobra continua sendo, para o modelo ajustado sem ela, uma observação nova. Repetindo o cálculo de lá parcela por parcela,

$$\mathbb{E}\big[\widehat{R}_{k\text{-CV}}\big] = R_{n-n/k}, \qquad \text{quando você queria } R_n.$$

Não há nada de misterioso no viés da validação cruzada: ele é o preço de avaliar um procedimento treinado com *menos dados* do que aquele que você vai usar no fim. Com $k = 10$ são 90% da amostra e quase não se nota; com $k = 2$, metade dela.

Duas consequências que vale guardar. A primeira é o **sinal**: como menos dados costumam piorar o ajuste, $R_{n-n/k} \geq R_n$, e a validação cruzada erra *por cima*. Quando houver viés, ele será pessimista — ao contrário do erro de treino, que erra sempre por baixo. A segunda é que esse viés não diz nada sobre a *variância*: aquela depende de quanto as dobras se parecem entre si, que é a outra metade do argumento clássico.

A Figura 3 mede o viés e o custo na nossa população.

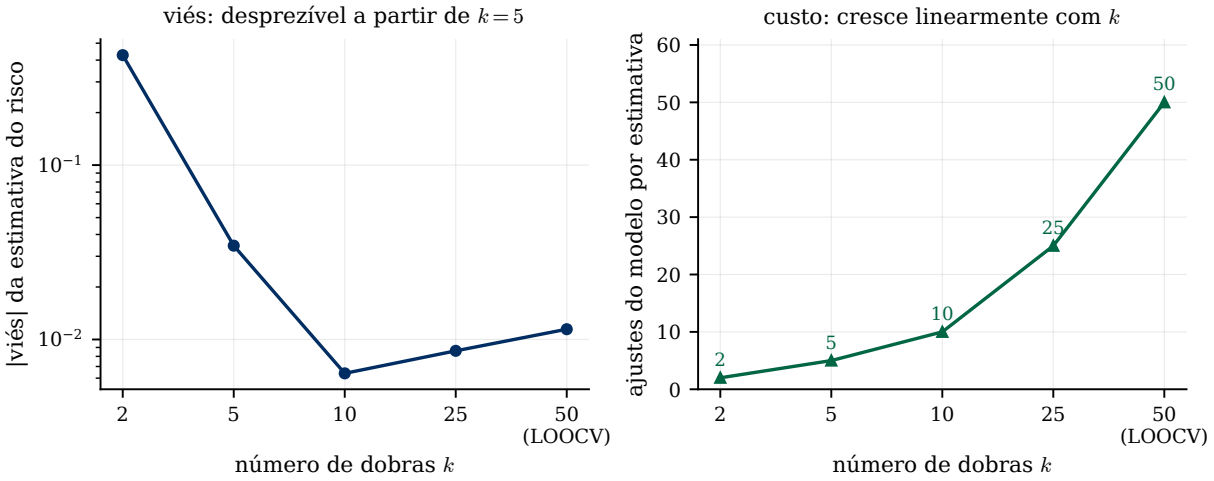


Figura 3: Estimativa do risco de um polinômio de grau 5 sobre $n = 50$ pontos, repetida em 300 amostras independentes. À esquerda, o viés da estimativa (escala logarítmica): com $k = 2$ ele é desastroso — treinar com 25 pontos é coisa muito diferente de treinar com 50 — e a partir de $k = 5$ já é desprezível. À direita, o custo, que cresce linearmente: a LOOCV pede *dez vezes* mais ajustes que $k = 5$.

Atenção: o que o experimento mostra

O viés se comporta como previsto: cai de 0,43 em $k = 2$ para 0,006 em $k = 10$ — treinar com metade dos dados é mesmo coisa muito diferente de treinar com todos, e essa diferença some depressa. Some tão depressa que a conclusão prática se lê direto do gráfico: de $k = 5$ para frente não há praticamente nada a ganhar, e o custo continua subindo linearmente. Daí $k = 5$ ou $k = 10$.

Atenção: A regra de ouro contra vazamento

Toda etapa que "aprende" algo dos dados — padronização, seleção de variáveis, escolha de λ — deve ocorrer *dentro* de cada dobra, usando apenas o treino daquela dobra. Padronizar com a média/desvio de todo o conjunto antes da CV é *data leakage* e produz estimativas otimistas. (É por isso que usamos *pipelines*, Aula 06.)

3.5 Selecionando hiperparâmetros

O uso central: para cada valor de um hiperparâmetro θ (p.ex. λ do Lasso), calcule $\widehat{R}_{k\text{-CV}}(\theta)$ e escolha $\widehat{\theta} = \arg\min_\theta \widehat{R}_{k\text{-CV}}(\theta)$.

Ideia central: o protocolo, em quatro passos

Junte as peças e o procedimento inteiro fica assim:

1. **separe** o conjunto de teste (p.ex. 30%) e guarde-o;
2. **escolha** um ou mais bons candidatos por validação cruzada *dentro do treino*;

3. **reajuste** o(s) candidato(s) em *todo* o conjunto de treinamento — as dobras serviram para escolher, não para entregar o modelo;
4. **meça** o desempenho no conjunto de teste.

O passo 3 é o que quase todo mundo esquece, e é ele que faz a CV valer a pena: o modelo entregue usa *todas* as observações de treino, não $k-1$ delas.

Atenção: Regra "1-EP"

Como $\widehat{R}_{k\text{-CV}}$ tem erro padrão, muitas vezes prefere-se a *regra de um erro-padrão*: escolher o modelo *mais simples* cujo risco estimado esteja a no máximo um erro-padrão do mínimo. Favorece parcimônia (é o `lambda.1se` do `glmnet`). Na Figura 2 ela escolheria o grau 3 em vez do 4: como as barras de erro dos graus 3 a 6 se sobrepõem, os dados não distinguem esses modelos, e nesse empate a regra manda ficar com o mais simples.

Atenção: Validação *aninhada*

Se você usa CV para *escolher* θ *e* para *reportar* o desempenho, precisa de **CV aninhada**: um laço externo estima o desempenho, um laço interno escolhe θ. Usar a mesma CV para as duas coisas subestima o risco — é o mesmo pecado do erro de treino, um andar acima. O [AME] descreve a versão com três conjuntos: *treino* ajusta, *validação* escolhe o hiperparâmetro, e o *teste*, tocado uma única vez no fim, estima o risco do vencedor. Quanto mais modelos você comparar na validação, mais otimista ela fica — por isso o teste existe.

4 O bootstrap

A validação cruzada estima o *risco*; o **bootstrap** estima a *incerteza* (variância, viés, intervalos) de praticamente qualquer estatística $\widehat{\theta}$ calculada a partir dos dados.

Ideia central

Não conhecemos P, mas temos a amostra. O bootstrap usa a *distribuição empírica* (reamostrar dos dados *com reposição*) como substituta de P, imitando o ato de coletar novas amostras.

Algoritmo. Para $b = 1, \ldots, B$:

1. sorteie n observações *com reposição* da amostra original (amostra bootstrap $\mathcal{D}^{*b}$);
2. calcule a estatística $\widehat{\theta}^{*b}$ em $\mathcal{D}^{*b}$.

A dispersão de $\{\widehat{\theta}^{*1}, \ldots, \widehat{\theta}^{*B}\}$ estima a distribuição amostral de $\widehat{\theta}$; por exemplo, $\widehat{\text{EP}}(\widehat{\theta})$ é o desvio-padrão dessas réplicas, e um intervalo de confiança de 95% pode ser dado pelos percentis 2,5% e 97,5%.

Exemplo 4.1 (O exemplo do [ISLP]: como dividir um investimento)**.** Você quer investir em dois ativos com retornos aleatórios X e Y, pondo uma fração α em X e $1-\alpha$ em Y. Para minimizar o risco da carteira, $\text{Var}(\alpha X + (1-\alpha)Y)$, a fração ótima é

$$\alpha = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}}.$$

Na prática você não conhece σ_X^2, σ_Y^2 e σ_{XY}: estima-os dos dados e obtém $\widehat{\alpha}$. Qual o erro-padrão de $\widehat{\alpha}$? Boa sorte tentando derivá-lo analiticamente — é uma razão de estimativas correlacionadas. Com bootstrap são cinco linhas de código: reamostre, recalcule $\widehat{\alpha}^{*b}$, tome o desvio-padrão. É aí

que está a força do método: funciona para estatísticas cuja distribuição amostral ninguém sabe escrever.

Observação 4.2. Cada amostra bootstrap deixa de fora, em média, $\approx 1/e \approx 37\%$ das observações originais (as *out-of-bag*). A conta: uma observação específica escapa de um sorteio com probabilidade $1 - 1/n$, e de n sorteios com probabilidade $(1 - 1/n)^n \to e^{-1} \approx 0{,}368$. Essa ideia reaparece no *bagging* e nas florestas aleatórias, onde as observações OOB dão uma estimativa de risco "de graça" — sem separar dados nem rodar CV.

5 Prática em Python

```python
from sklearn.model_selection import (train_test_split, cross_val_score, KFold,
                                     GridSearchCV)
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error

# 0. O PRIMEIRO passo e separar o teste. Dobra nenhuma vai toca-lo.
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)

modelo = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
dobras = KFold(5, shuffle=True, random_state=0)

# 1. CV DENTRO DO TREINO, para estimar o risco de UM procedimento
# (MSE -> negativo no sklearn)
scores = cross_val_score(modelo, X_tr, y_tr, cv=dobras,
                         scoring="neg_mean_squared_error")
print("risco estimado:", -scores.mean(), "+/-", scores.std())

# 2. Selecionar o hiperparametro lambda (=alpha) por CV, SEM vazamento
# (o StandardScaler e reajustado dentro de cada dobra)
grade = {"ridge__alpha": [0.01, 0.1, 1, 10, 100]}
busca = GridSearchCV(modelo, grade, cv=dobras, scoring="neg_mean_squared_error")
busca.fit(X_tr, y_tr) # o teste continua guardado
print("melhor alpha:", busca.best_params_)

# 3. O vencedor ja saiu reajustado em TODO o treino (refit=True, o padrao).
# So agora o teste e tocado, uma unica vez.
print("EQM no teste:", mean_squared_error(y_te, busca.predict(X_te)))
```

Atenção

Três coisas a reparar nesse trecho. Primeira: as funções de *score* seguem a convenção "maior é melhor", então o EQM aparece negado (`neg_mean_squared_error`) — daí o sinal de menos ao imprimir. Segunda: o `KFold` está construído explicitamente, e não passado como `cv=5` — assim é passado `shuffle=True`, sem o qual as dobras saem *na ordem do arquivo*, com o problema descrito no início da aula. Terceira: o `X_te` aparece uma única vez no trecho inteiro, na última linha. Se ele aparecer antes disso, alguma coisa está errada.

O notebook `Aula prática 03` refaz tudo isto: implementa as k dobras à mão, confere o atalho do LOOCV, roda o protocolo de quatro passos numa população em que a resposta é conhecida, e fecha escolhendo o λ de uma Ridge no `superconductivity.csv`.

Resumo

- O erro de treino é otimista; estimar o risco exige dados *não* usados no ajuste.
- *Data splitting*: simples, mas desperdiça dados e é instável.
- *LOOCV* (3): quase não-viesado; atalho fechado para modelos lineares (Prop. 3.2).
- *k-dobras* (4): usa toda a amostra (Fig. 1); $k = 5$ ou 10 bastam — além disso só cresce o custo (Fig. 3).
- A CV pode errar o *nível* do risco e ainda assim acertar *onde* está o mínimo (Fig. 2) — e é isso que a seleção de modelos precisa.
- Use CV para escolher *tuning parameters*; cuidado com *vazamento* (faça tudo dentro da dobra) e com CV aninhada para reportar desempenho.
- *Bootstrap*: reamostragem com reposição para medir incerteza de estimativas; conecta com OOB e *bagging*.

Leitura recomendada. [AME] §1.4 (função de risco e Teorema 1), §1.5.1 (*data splitting*, validação cruzada, a Observação 1.5 sobre o LOOCV não-viesado e o intervalo de confiança para o risco). [ISLP] Capítulo 5: §5.1 (validação cruzada — vale muito olhar as Figuras 5.6 e 5.8, que são a versão deles da nossa Figura 2), §5.1.4 (o argumento viés–variância para a escolha de k) e §5.2 (*bootstrap*, com o exemplo do investimento).

Para praticar. `Lista teorica 03.pdf` (teórica, com gabarito) e `Lista prática 03.ipynb` (prática, para completar as lacunas), nesta mesma pasta.